



Escuela de posgrado
Universidad Autónoma del Perú

PROGRAMA ACADÉMICO DE MAESTRÍA EN INGENIERIA ARTIFICIAL

TÍTULO DEL PROYECTO

RiskScore OECE

Modelo predictivo binario para estimar el riesgo de sanción futura de proveedores adjudicados

AUTORES:

Fátima Toche Vega ([orcid.org/ 0000-0002-1984-4954](https://orcid.org/0000-0002-1984-4954))

Diego Avendaño Díaz

DOCENTE:

Luis Arnaldo Godoy Fuentes

LÍNEA DE DESARROLLO:

Inteligencia Artificial y Ciencia de Datos

LIMA — PERÚ

2026

ÍNDICE DE CONTENIDOS

I. INTRODUCCIÓN.....	5
II. DEFINICIÓN DE PROBLEMA DEL NEGOCIO	5
2.1. Contexto y problema de negocio	5
2.2. Objetivo del proyecto de IA	6
2.3. KPI de negocio	7
III. DATOS Y DIAGNÓSTICO INICIAL	9
3.1. Fuente de los datos	9
3.2. Descripción del dataset	9
3.3. Columnas principales	10
3.4. Variable objetivo	12
3.5. Problemas en calidad de datos	14
3.6. Dificultades encontradas	16
IV. PIPELINE PROPUESTO Y MÓDULOS DEL PROYECTO	17
4.1. Descripción breve del Pipeline	17
4.2. Diagrama de flujo de las etapas del proyecto.....	18
4.3. Tabla de módulos Python y relación con las etapas	20
4.4. Resumen del blueprint de módulos	23

Módulo 1: Carga de datos	23
Módulo 2: Limpieza y normalización	23
Módulo 3: Construcción de la variable objetivo	24
Módulo 4: Ingeniería de variables	24
Módulo 5: Análisis exploratorio	24
Módulo 6: Entrenamiento del modelo	25
Módulo 7: Evaluación del modelo.....	25
Módulo 8: Comunicación de resultados	26
V. DESARROLLO Y EJECUCIÓN DEL PROYECTO.....	26
5.1. Fase 1: Identificar el CSV y la variable objetivo	27
5.2. Fase 2: Cargar los datos correctamente	29
5.3. Fase 3: Realizar limpieza básica	30
5.4. Fase 4: Creación de Features	32
5.5. Fase 5: Entrenamiento del modelo.....	37
5.6. Fase 6: Evaluación del modelo	45
5.7. Fase 7: Guardar el Pipeline completo en .pkl & .joblib	48
Conclusión del capítulo	51
ANEXOS	52
Anexo 1: Problema de negocio y KPI alineado	52
Anexo 2: Datos necesarios, fuentes, volumetría y frecuencia	53

I. INTRODUCCIÓN

La contratación pública peruana moviliza un volumen significativo de recursos y requiere que las entidades públicas gestionen riesgos asociados al desempeño, cumplimiento normativo y antecedentes de sus proveedores. En este contexto, los registros históricos de adjudicaciones y sanciones administrativas permiten construir herramientas analíticas para anticipar riesgos y priorizar revisiones preventivas.

El presente proyecto propone el desarrollo de un modelo predictivo binario denominado RiskScore OECE. El modelo busca estimar si un proveedor adjudicado será sancionado posteriormente dentro de un horizonte de 24 meses contado desde la fecha de buena pro. Esta formulación transforma un problema de revisión reactiva de antecedentes en un problema de evaluación preventiva del riesgo regulatorio futuro.

La utilidad de la inteligencia artificial se justifica porque la cantidad de adjudicaciones y proveedores dificulta una revisión manual exhaustiva. Mediante aprendizaje automático supervisado, el sistema puede aprender patrones históricos asociados a sanciones futuras, tales como características del proveedor, montos adjudicados, tipo de procedimiento, objeto contractual, entidad contratante e historial previo de adjudicaciones o sanciones.

El enfoque tiene valor para entidades públicas que desean fortalecer sus controles, empresas privadas que realizan due diligence de proveedores, bancos que financian contratistas del Estado, aseguradoras que emiten cartas fianza o pólizas de caución, y consultoras de cumplimiento que ofrecen evaluación de riesgos a terceros.

II. DEFINICIÓN DE PROBLEMA DEL NEGOCIO

2.1. Contexto y problema de negocio

Actualmente, la evaluación de proveedores en contratación pública suele centrarse en verificar si una empresa ya se encuentra sancionada o impedida. Este enfoque es necesario, pero insuficiente para una gestión moderna del riesgo, porque no permite estimar la probabilidad de que un proveedor adjudicado sea sancionado en el futuro próximo.

El problema de negocio consiste en que las entidades y organizaciones que contratan, financian o aseguran proveedores del Estado no siempre cuentan con una herramienta objetiva para priorizar qué proveedores o adjudicaciones requieren revisión reforzada. Como resultado, pueden producirse costos por incumplimientos, demoras contractuales, sobrecostos, pérdida reputacional o exposición financiera.

La oportunidad de mejora consiste en utilizar datos históricos de adjudicaciones y sanciones para construir un score de riesgo que ayude a ordenar los casos según su probabilidad de sanción futura. El modelo no reemplaza la evaluación legal ni administrativa, sino que funciona como herramienta de apoyo para priorizar recursos de control y análisis.

2.2. Objetivo del proyecto de IA

Objetivo general:

Desarrollar un modelo de machine learning supervisado de clasificación binaria que estime la probabilidad de que un proveedor adjudicado en una contratación pública sea sancionado dentro de los 24 meses posteriores a la fecha de buena pro.

Objetivos específicos:

- Integrar las bases de adjudicaciones CONOECE 2018-2026 con el registro de proveedores sancionados del OECE utilizando el RUC como identificador principal.

- Construir una variable objetivo binaria denominada SANCIONADO_24M, evitando confundirla con reincidencia.
- Generar variables predictoras disponibles al momento de la adjudicación, incluyendo características del procedimiento, del proveedor y de su historial previo.
- Diagnosticar problemas de calidad de datos, desbalance de clases y limitaciones temporales del horizonte de observación.
- Proponer KPI de negocio y métricas técnicas que permitan evaluar la utilidad del modelo para la toma de decisiones.

2.3. KPI de negocio

Los KPI de negocio deben medir el impacto operativo y preventivo del modelo. Dado que el evento de sanción futura es poco frecuente, la evaluación no debe concentrarse solamente en accuracy, sino en la capacidad de identificar oportunamente los casos de mayor riesgo.

KPI	Definición	Uso para el negocio
Tasa de detección de sancionados futuros	Porcentaje de proveedores que fueron sancionados dentro de 24 meses y fueron clasificados como riesgo alto.	Mide la capacidad preventiva del modelo.
Recall de la clase positiva	Casos SANCIONADO_24M = 1 correctamente identificados entre todos los casos positivos.	Prioriza no dejar pasar proveedores riesgosos.

Precisión en proveedores de alto riesgo	Proporción de alertas de alto riesgo que efectivamente terminan sancionadas.	Evita alertas excesivas y revisiones innecesarias.
Monto adjudicado expuesto a riesgo	Suma de montos adjudicados en casos clasificados como alto riesgo.	Traduce el riesgo a impacto económico.
Reducción del universo de revisión manual	Porcentaje de adjudicaciones priorizadas para revisión frente al total.	Mide eficiencia operativa en due diligence.
Lift del decil superior	Concentración de casos positivos en el 10 % de adjudicaciones con mayor score.	Evalúa si el modelo ordena adecuadamente los riesgos.

III. DATOS Y DIAGNÓSTICO INICIAL

3.1. Fuente de los datos

El proyecto utiliza dos fuentes principales de datos abiertos vinculadas a contratación pública peruana:

- Base de adjudicaciones CONOECE 2018-2026: contiene información de procedimientos de selección, entidades contratantes, proveedores, montos adjudicados, objeto contractual y fechas relevantes del proceso.
- Registro de proveedores sancionados del OECE: contiene RUC, razón social, fechas de inicio y fin de sanción, número de resolución, motivo de infracción y descripción del motivo.

La integración de ambas fuentes permite convertir un registro histórico de contratación y sanciones en una base modelable para aprendizaje supervisado. El cruce se realiza mediante el RUC del proveedor, con normalización previa del identificador.

3.2. Descripción del dataset

Luego de la integración y depuración inicial, la base modelable contiene 385,138 registros de adjudicaciones, 133,581 proveedores únicos y una ventana temporal de buena pro entre el 03/01/2018 y el 20/06/2024. Cada fila representa una adjudicación o ítem adjudicado a un proveedor.

La unidad de análisis es la adjudicación, no la empresa sancionada. Esta precisión es importante: el modelo no se entrena solo con proveedores sancionados, sino con proveedores adjudicados que luego pueden o no aparecer sancionados dentro del horizonte definido.

Indicador	Resultado
-----------	-----------

Registros modelables	385,138
Proveedores únicos	133,581
Periodo de buena pro usado	03/01/2018 a 20/06/2024
Casos positivos SANCIONADO_24M = 1	9,782
Casos negativos SANCIONADO_24M = 0	375,356
Tasa positiva observada	2.54%

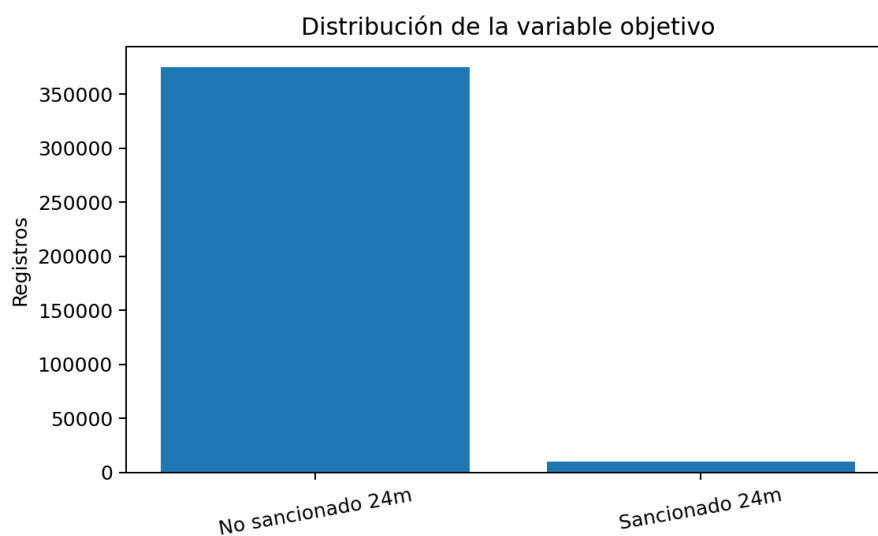


Figura 1. Distribución de la variable objetivo SANCIONADO_24M.

3.3. Columnas principales

Las columnas principales se agrupan en variables identificadoras, variables del procedimiento, variables económicas, variables temporales, variables geográficas, variables de historial y variable objetivo.

Grupo	Columnas	Descripción
Identificación del proveedor	ruc_clean, proveedor, tipo_proveedor	Identifican al adjudicatario y su tipo de proveedor.
Características del procedimiento	objetocontractual, tipoprocesoseleccion, estado_item	Describen la naturaleza de la contratación y el tipo de proceso.
Variables económicas	monto_adjudicado_item_soles, monto_referencial_item_soles, cantidad_adjudicado_item, ratio_monto_adjudicado_referencial	Miden exposición económica y relación entre monto adjudicado y referencial.
Variables geográficas e institucionales	departamento_item, entidad, tipoentidad, entidad_departamento	Describen ubicación del objeto contractual y entidad contratante.
Variables temporales	fecha_buenapro, anio_buenapro, mes_buenapro	Permiten ordenar eventos y construir el horizonte de predicción.
Historial previo del proveedor	n_adjudicaciones_previas, monto_adjudicado_acum_previo, monto_promedio_previo, sanciones_previas, tuvo_sancion_previa	Resumen de experiencia y antecedentes disponibles antes de la buena pro.
Variable objetivo	SANCIONADO_24M	Indica si hubo al menos una sanción posterior dentro de 24 meses.

Para evitar fuga de información, las variables de historial deben calcularse únicamente con información disponible antes de la fecha de buena pro. Por ejemplo, sanciones_previas solo debe contar sanciones cuya fecha de inicio sea anterior a la adjudicación analizada.

3.4. Variable objetivo

La variable objetivo del modelo es SANCIONADO_24M. Esta variable es binaria y responde a la pregunta: ¿el proveedor adjudicado registra al menos una sanción posterior dentro de los 24 meses siguientes a la fecha de buena pro?

El valor 1 no significa que el proveedor haya tenido dos sanciones, ni que sea reincidente. En este enfoque actualizado, el modelo ya no predice reincidencia entre empresas previamente sancionadas. El modelo predice sanción futura de proveedores adjudicados.

Valor de SANCIONADO_24M	Interpretación correcta	Interpretación incorrecta que debe evitarse
1	El proveedor adjudicado recibió al menos una sanción posterior dentro de los 24 meses siguientes a la buena pro.	No significa necesariamente que tuvo dos sanciones. No significa reincidencia.
0	El proveedor adjudicado no registra sanción posterior dentro de los 24 meses siguientes a la buena pro.	No significa que nunca haya sido sancionado; solo que no fue sancionado dentro de esa ventana posterior.

Regla de construcción:

Para cada adjudicación, se compara la fecha_buenapro con las fechas de inicio de sanción del mismo RUC. Si existe al menos una sanción con fecha posterior a la buena pro y menor o igual a 24 meses, entonces SANCIONADO_24M = 1. En caso contrario, SANCIONADO_24M = 0.

SANCIONADO_24M = 1 si existe FECHA_INICIO_SANCIÓN en (fecha_buenapro, fecha_buenapro + 24 meses]

Fecha buena pro	Fecha de sanción posterior observada	Resultado	Explicación
10/03/2021	15/12/2021	1	Existe una sanción posterior dentro de 24 meses.
10/03/2021	Sin sanción posterior	0	No existe sanción posterior en la ventana.
10/03/2021	20/08/2024	0	La sanción ocurre fuera de los 24 meses.
10/03/2021	01/02/2021	0	La sanción es anterior a la buena pro; podría usarse como antecedente, pero no como target futuro.

Esta definición convierte el problema en clasificación binaria: el modelo aprende a distinguir adjudicaciones con riesgo observado de sanción futura frente a adjudicaciones sin sanción futura observada en el horizonte de 24 meses.

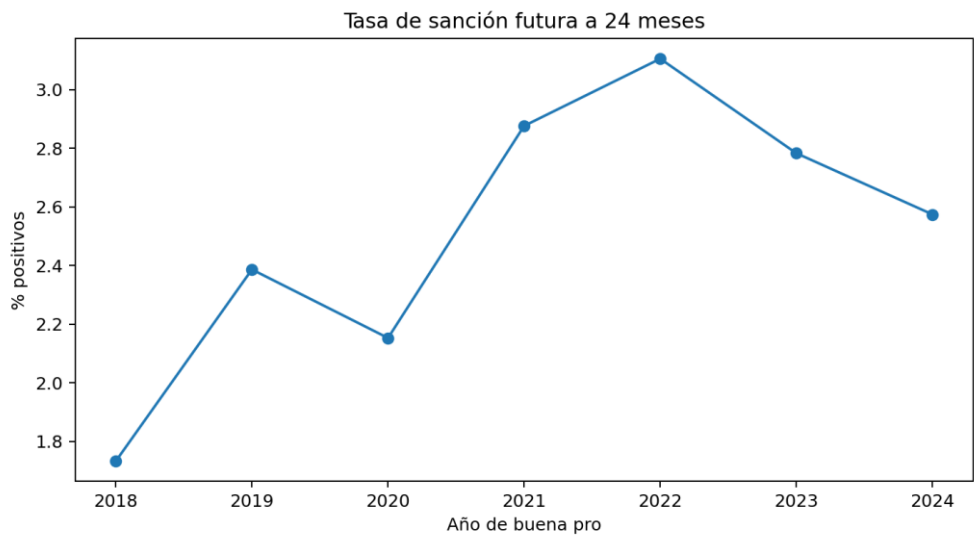


Figura 2. Tasa observada de sanción futura por año de buena pro.

3.5. Problemas en calidad de datos

Durante la preparación del dataset se identificaron los siguientes problemas de calidad de datos:

Problema	Descripción	Tratamiento propuesto
Normalización de RUC	Existen proveedores no domiciliados o identificadores que no siguen el formato peruano estándar de 11 dígitos.	Estandarizar como texto, conservar ceros iniciales y analizar separadamente los casos no domiciliados.

Fechas incompletas o inconsistentes	Algunas fechas pueden estar ausentes o con formatos distintos entre fuentes.	Convertir a formato fecha, validar rangos y excluir registros sin fecha_buenapro para el entrenamiento.
Ventana temporal incompleta	Adjudicaciones recientes no tienen 24 meses completos para observar sanción futura.	Entrenar solo con adjudicaciones con ventana completa o marcar datos recientes como no aptos para entrenamiento.
Clase desbalanceada	Solo 2.54% de registros tiene SANCIONADO_24M = 1.	Usar métricas como recall, PR-AUC, F1 y lift; considerar class_weight o técnicas de balanceo.
Riesgo de fuga de información	Variables posteriores a la buena pro podrían filtrar el resultado futuro.	Construir variables predictoras solo con información disponible hasta la fecha_buenapro.
Duplicidad por ítems	Un mismo proveedor puede aparecer varias veces por ítems o procesos.	Definir claramente la unidad de análisis como adjudicación o ítem adjudicado.
Valores faltantes	Se observan campos con valores vacíos, especialmente ubicación, monto referencial o tipo de proveedor.	Imputación simple, categoría desconocido o exclusión según relevancia.

3.6. Dificultades encontradas

Las principales dificultades metodológicas se relacionan con la integración de fuentes, el diseño de la variable objetivo y la interpretación correcta del resultado del modelo.

- La base de sancionados por sí sola no permite predecir sancionado vs. no sancionado, porque contiene únicamente empresas sancionadas. Por ello se incorporó la base de adjudicaciones para obtener casos negativos observables.
- El cruce por RUC requiere normalización cuidadosa, especialmente en proveedores no domiciliados, consorcios o registros con identificadores no estándar.
- Debe evitarse interpretar $\text{SANCIONADO}_{24M} = 1$ como reincidencia. El target representa al menos una sanción futura dentro del horizonte de 24 meses.
- La sanción futura puede no estar causalmente vinculada con la adjudicación específica analizada. Por ello el modelo debe presentarse como estimador de riesgo regulatorio futuro del proveedor, no como prueba de incumplimiento en un contrato concreto.
- El desbalance de clases obliga a priorizar métricas orientadas a la clase positiva y a explicar por qué accuracy puede ser engañosa.
- Las adjudicaciones de 2025 y 2026 deben usarse con cautela para entrenamiento del target de 24 meses, porque aún no se ha completado la ventana de observación.

IV. PIPELINE PROPUESTO Y MÓDULOS DEL PROYECTO

4.1. Descripción breve del Pipeline

El proyecto propone un pipeline de ciencia de datos orientado a construir un modelo de machine learning de clasificación binaria para estimar el riesgo de sanción futura de proveedores adjudicados en contrataciones públicas peruanas. La variable objetivo del modelo será SANCIONADO_24M, donde el valor 1 indica que el proveedor recibió al menos una sanción dentro de los 24 meses posteriores a la fecha de buena pro, y el valor 0 indica que no registra sanción posterior dentro de dicho horizonte temporal.

El pipeline se estructura de manera secuencial, desde la ingesta de datos hasta la generación de resultados interpretables para la toma de decisiones. En primer lugar, se integran las bases de adjudicaciones públicas del periodo 2018-2026 con la base de empresas sancionadas del OECE. Luego, se realiza la limpieza, estandarización y validación de los datos, con especial atención al uso del RUC como identificador principal del proveedor.

Posteriormente, se construye la variable objetivo mediante el cruce entre la fecha de buena pro y las fechas de inicio de sanción. En esta etapa se identifica si una adjudicación fue seguida por una sanción al mismo proveedor dentro de los 24 meses posteriores. Luego, se generan variables predictoras a partir de información disponible al momento de la adjudicación, tales como monto adjudicado, tipo de proceso, objeto contractual, tipo de proveedor, entidad contratante, departamento y antecedentes previos del proveedor.

Una vez construida la base modelable, se realiza el análisis exploratorio de datos para identificar patrones, valores atípicos, distribución de clases, variables relevantes y posibles problemas de calidad. Después, se divide la información en conjuntos de entrenamiento y prueba, procurando evitar fuga de información temporal. Sobre esta base, se entrenan modelos de clasificación binaria, iniciando con modelos interpretables como regresión logística

y comparándolos con modelos de mayor capacidad predictiva como Random Forest o Gradient Boosting.

Finalmente, el modelo se evalúa mediante métricas técnicas y de negocio. Debido al probable desbalance de clases, la evaluación no se centrará únicamente en accuracy, sino en métricas como recall, precision, F1-score, ROC-AUC, PR-AUC y capacidad de detección de proveedores sancionados en los segmentos de mayor riesgo. El resultado esperado es un score de riesgo que permita priorizar revisiones, fortalecer procesos de due diligence y reducir la exposición de entidades públicas, bancos, aseguradoras o empresas privadas frente a proveedores con mayor probabilidad de sanción futura.

4.2. Diagrama de flujo de las etapas del proyecto

El pipeline propuesto puede representarse en las siguientes etapas:



El flujo evidencia que el proyecto no se limita a entrenar un modelo estadístico, sino que parte de un problema de negocio, transforma fuentes públicas en una base analítica y genera una herramienta de apoyo a la decisión. El resultado final no debe entenderse como una predicción determinista ni como una atribución causal de responsabilidad, sino como una estimación probabilística de riesgo regulatorio futuro.

4.3. Tabla de módulos Python y relación con las etapas

Etapas del pipeline	Módulo Python propuesto	Función principal dentro del proyecto
Ingesta de datos	pandas	Leer archivos Excel y CSV, consolidar bases anuales de adjudicaciones y cargar la base de sancionados.
Ingesta de datos	openpyxl	Permitir la lectura de archivos Excel de adjudicaciones en formato .xlsx.
Limpieza de datos	pandas	Estandarizar nombres de columnas, eliminar duplicados, tratar valores nulos y transformar tipos de datos.
Limpieza de datos	numpy	Realizar operaciones numéricas, creación de variables y tratamiento de valores faltantes.
Tratamiento de fechas	datetime	Convertir y comparar fechas de buena pro, fechas de convocatoria y fechas de sanción.
Validación de identificadores	re	Validar formato de RUC, identificar registros con caracteres no válidos y limpiar cadenas de texto.
Integración de fuentes	pandas.merge	Cruzar adjudicaciones y sanciones usando el RUC como llave principal.

Etapa del pipeline	Módulo Python propuesto	Función principal dentro del proyecto
Construcción del target	pandas, numpy	Crear la variable binaria SANCIONADO_24M según la existencia de una sanción posterior dentro de 24 meses.
Ingeniería de variables	pandas	Crear variables históricas del proveedor, agregados de montos, número de adjudicaciones previas y antecedentes de sanción.
Análisis exploratorio	matplotlib	Elaborar gráficos de distribución, frecuencia, montos adjudicados y proporción de casos positivos.
Análisis exploratorio	seaborn	Visualizar patrones por categorías, mapas de calor y relaciones entre variables.
Preprocesamiento	sklearn.preprocessing	Codificar variables categóricas, escalar variables numéricas y preparar datos para modelado.
División de datos	sklearn.model_selection	Separar el dataset en entrenamiento y prueba, idealmente respetando un enfoque temporal.
Modelado base	sklearn.linear_model	Entrenar una regresión logística como modelo interpretable de referencia.

Etapa del pipeline	Módulo Python propuesto	Función principal dentro del proyecto
Modelado avanzado	sklearn.ensemble	Entrenar modelos como Random Forest o Gradient Boosting para mejorar desempeño predictivo.
Evaluación	sklearn.metrics	Calcular recall, precision, F1-score, ROC-AUC, matriz de confusión y curvas de evaluación.
Interpretabilidad	sklearn.inspection	Evaluar importancia de variables y analizar los factores que explican el score de riesgo.
Exportación de resultados	pandas	Exportar la base modelable, predicciones, scores de riesgo y tablas de resultados en CSV o Excel.

La selección de módulos responde a las necesidades del proyecto: pandas constituye el eje central para la preparación de datos; numpy y datetime permiten transformar variables y construir el horizonte temporal del target; scikit-learn permite implementar modelos de clasificación binaria; y las librerías de visualización permiten comunicar hallazgos de manera clara en el informe y en la presentación.

4.4. Resumen del blueprint de módulos

El blueprint técnico del proyecto organiza el trabajo en módulos funcionales. Esta separación facilita que el desarrollo sea ordenado, reproducible y escalable. Cada módulo cumple una función específica y genera salidas que alimentan la siguiente etapa del pipeline.

Módulo 1: Carga de datos

Este módulo importa las bases de adjudicaciones anuales y la base de sancionados. Las adjudicaciones provienen de archivos Excel del periodo 2018-2026, mientras que las sanciones provienen de un archivo CSV. La salida de este módulo será una base consolidada de adjudicaciones y una base normalizada de sanciones.

- Archivos CONOECE_ADJUDICACIONES2018_0.xlsx hasta CONOECE_ADJUDICACIONES2026_0.xlsx.
- Archivo sancionados.csv.
- Salidas esperadas: adjudicaciones_consolidadas y sancionados_normalizado.

Módulo 2: Limpieza y normalización

Este módulo transforma los datos originales para hacerlos utilizables en el análisis. Incluye la estandarización de nombres de columnas, conversión de fechas, validación de RUC, tratamiento de montos, revisión de nulos y depuración de registros inconsistentes.

- Normalizar el RUC del proveedor.
- Convertir fechas a formato fecha.
- Convertir montos adjudicados a formato numérico.
- Identificar registros con datos incompletos.
- Excluir o marcar registros que no puedan cruzarse adecuadamente.

Módulo 3: Construcción de la variable objetivo

Este módulo construye la variable binaria SANCIONADO_24M. Para cada adjudicación, se verifica si el proveedor recibió una sanción posterior dentro de los 24 meses siguientes a la fecha de buena pro. Si existe al menos una sanción posterior en ese periodo, el valor será 1. Si no existe sanción en ese horizonte, el valor será 0.

- SANCIONADO_24M = 1 si existe una FECHA_INICIO de sanción del mismo RUC posterior a la FECHA_BUENAPRO y menor o igual a 24 meses después de dicha fecha.
- SANCIONADO_24M = 0 si no existe sanción posterior dentro de ese periodo.
- Precisión metodológica: SANCIONADO_24M = 1 no significa que el proveedor tenga dos sanciones ni que sea reincidente; significa al menos una sanción futura dentro del horizonte temporal definido.

Módulo 4: Ingeniería de variables

Este módulo crea variables predictoras relevantes para el modelo. Se utilizarán variables disponibles en la adjudicación, así como variables históricas del proveedor calculadas únicamente con información anterior a la fecha de buena pro, para evitar fuga de información.

- monto_adjudicado_item_soles, objeto_contractual, tipo_proceso_seleccion y tipo_proveedor.
- departamento_item, tipo_entidad, anio_buenapro y mes_buenapro.
- n_adjudicaciones_previas, monto_adjudicado_acumulado_previo, n_entidades_previas, tuvo_sancion_previa y n_sanciones_previas.
- ratio_monto_adjudicado_referencial.

Módulo 5: Análisis exploratorio

Este módulo permite comprender la estructura del dataset antes del modelado. Se analizará la distribución de adjudicaciones por año, el porcentaje de casos positivos, la concentración

por tipo de proceso, el comportamiento de los montos adjudicados, la distribución geográfica y las diferencias entre adjudicaciones con y sin sanción posterior.

- ¿Qué proporción de adjudicaciones termina asociada a una sanción futura?
- ¿Existen diferencias por objeto contractual o tipo de procedimiento?
- ¿El monto adjudicado se relaciona con mayor exposición al riesgo?
- ¿Existe concentración de riesgo en ciertos departamentos o tipos de entidad?

Módulo 6: Entrenamiento del modelo

Este módulo entrena modelos de clasificación binaria. Se recomienda iniciar con una regresión logística como modelo base por su interpretabilidad. Luego, se pueden probar modelos más flexibles como Random Forest o Gradient Boosting para mejorar el desempeño predictivo.

- Modelos candidatos: regresión logística, árbol de decisión, Random Forest y Gradient Boosting.
- Salidas esperadas: modelo entrenado, predicciones de probabilidad y clasificación de riesgo bajo, medio y alto.

Módulo 7: Evaluación del modelo

Este módulo evalúa el desempeño del modelo desde una perspectiva técnica y de negocio. Dado que la clase positiva probablemente representa una proporción menor del total, no debe utilizarse accuracy como métrica principal. Se recomienda priorizar recall, precision, F1-score, PR-AUC y lift en los segmentos de mayor riesgo.

- Métricas sugeridas: recall, precision, F1-score, ROC-AUC, PR-AUC y matriz de confusión.
- Indicadores de negocio: porcentaje de sancionados capturados en el top 10 % de riesgo y monto adjudicado expuesto a proveedores clasificados como alto riesgo.

Módulo 8: Comunicación de resultados

Este módulo convierte los resultados técnicos en información útil para la toma de decisiones.

El resultado final debe permitir que un usuario de negocio identifique proveedores o adjudicaciones con mayor probabilidad de sanción futura y priorice acciones de revisión, seguimiento o due diligence.

- Score de riesgo por adjudicación.
- Segmentación de riesgo: bajo, medio y alto.
- Tabla de principales variables asociadas al riesgo.
- Recomendaciones de uso del modelo y visualizaciones para la sustentación.

En conjunto, el blueprint permite pasar de datos públicos dispersos a una solución analítica orientada al negocio. El valor del proyecto no está solo en la predicción, sino en transformar los datos históricos de contratación pública y sanciones en una herramienta preventiva para la gestión de riesgos.

V. DESARROLLO Y EJECUCIÓN DEL PROYECTO

El presente capítulo documenta la ejecución práctica del proyecto RiskScore OSCE en Anaconda y Jupyter Notebook. El flujo se organizó en siete fases sucesivas: identificación del archivo CSV y de la variable objetivo, carga de datos, limpieza básica, creación de features, entrenamiento del modelo, evaluación y guardado del pipeline. La finalidad técnica del proyecto es construir un modelo de clasificación binaria que estime el riesgo de que un proveedor registre una sanción posterior dentro de una ventana de 24 meses, usando como variable objetivo `target_sancionado_24m`.

Fase	Actividad	Resultado principal
Fase 1	Identificación del CSV y target	CONOSCE_ADJUDICACIONES_2018_2026_CONSOLIDADO.csv y target_sancionado_24m
Fase 2	Carga de datos	Lectura con pandas, separador ;, codificación UTF-8 y control de tipos
Fase 3	Limpieza básica	Diagnóstico y tratamiento de nulos y duplicados
Fase 4	Feature engineering	Creación de variables temporales, de duración, RUC, motivo e historial
Fase 5	Entrenamiento	Regresión Logística balanceada como modelo base
Fase 6	Evaluación	Matriz de confusión, classification report, comparación de modelos e importancia de variables
Fase 7	Guardado del pipeline	Exportación .joblib, .pkl y artefacto con metadatos

5.1. Fase 1: Identificar el CSV y la variable objetivo

La primera fase consistió en ubicar el archivo CSV de trabajo dentro de la estructura del proyecto y precisar la variable objetivo que será utilizada en el modelo predictivo. El archivo base se encuentra en data/raw y corresponde a CONOSCE_ADJUDICACIONES_2018_2026_CONSOLIDADO.csv.

Para el modelamiento se definió la variable target_sancionado_24m. Esta variable tiene naturaleza binaria: el valor 1 representa proveedores que registran una sanción posterior dentro del horizonte de 24 meses; el valor 0 representa proveedores que no registran

sanción posterior en dicho periodo. Esta definición convierte el proyecto en un problema de clasificación binaria.

e equipo > Windows (C:) > Proyecto-Risk-Score-OECE >				
Ordenar Ver				
Nombre	Fecha de modificación	Tipo	Tamaño	
config	23/07/2026 11:17	Carpeta de archivos		
data	23/07/2026 11:19	Carpeta de archivos		
logs	23/07/2026 11:17	Carpeta de archivos		
models	1/08/2026 00:42	Carpeta de archivos		
notebooks	1/08/2026 00:44	Carpeta de archivos		
src	23/07/2026 11:17	Carpeta de archivos		
tests	23/07/2026 11:17	Carpeta de archivos		
environment.yml	23/07/2026 12:24	Archivo de origen ...	1 KB	
README.md	23/07/2026 11:54	Archivo de origen ...	1 KB	
requirements.txt	23/07/2026 12:23	Documento de tex...	3 KB	

e equipo > Windows (C:) > Proyecto-Risk-Score-OECE > data >				
Ordenar Ver				
Nombre	Fecha de modificación	Tipo	Tamaño	
processed	1/08/2026 00:33	Carpeta de archivos		
raw	23/07/2026 11:59	Carpeta de archivos		

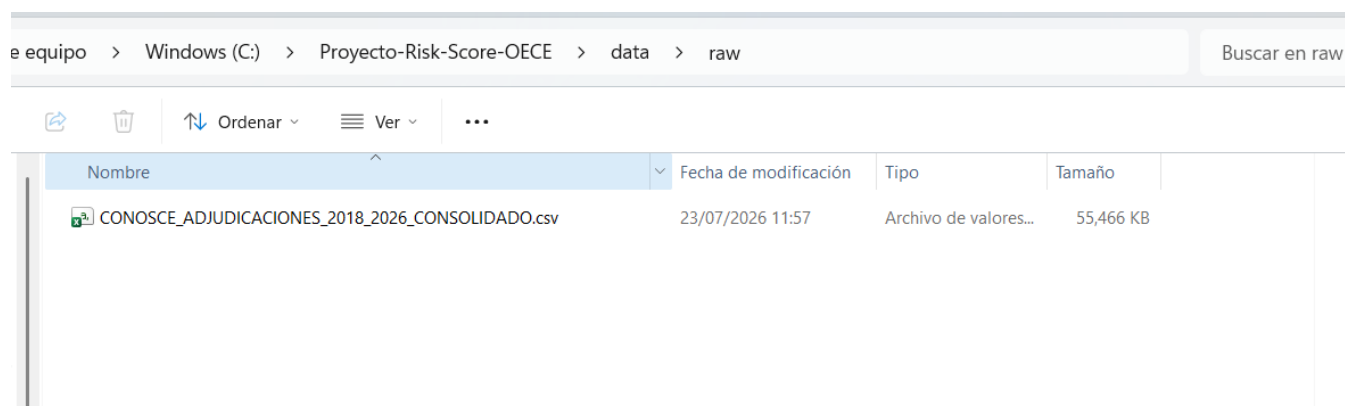


Figura 5.1. Identificación del CSV base y definición de la variable objetivo.

5.2. Fase 2: Cargar los datos correctamente

La segunda fase tuvo como finalidad cargar el dataset en Jupyter Notebook usando pandas. En la ejecución se utilizó `read_csv` con separador punto y coma, porque el archivo CSV no utiliza coma como delimitador. Asimismo, se cargaron los campos como texto para evitar problemas de formato, especialmente en identificadores como RUC, códigos de entidad o códigos de convocatoria.

Como evidencia de carga correcta, el notebook mostró las primeras filas, la dimensión del dataset y los tipos de datos. La base raw contenía 100,089 registros y 26 columnas, entre ellas: `anio`, `entidad`, `tipoentidad`, `objetocontractual`, `tipoprocesoseleccion`, `monto_adjudicado_item_soles`, `ruc_proveedor`, `proveedor` y `fecha_buenapro`.

```
jupyter 01_carga_datos Last Checkpoint: 8 days ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

=====
PRIMERAS 100 FILAS DEL DATASET:
=====
  anio  codigoentidad  entidad_ruc  \
0  2018         1906.0  2.014809e+10
1  2018          49.0  2.040633e+10
2  2018        10530.0  2.040845e+10
3  2018        1901.0  2.017495e+10
4  2018        1014.0  2.020598e+10

                                entidad  tipoentidad  \
0                UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS  GOBIERNO NACIONAL
1      GOBIERNO REGIONAL DE PUNO SEDE CENTRAL  GOBIERNO REGIONAL
2  GOBIERNO REGIONAL DE LORETO-SALUD LORETO Y PER...  GOBIERNO REGIONAL
3  UNIVERSIDAD NACIONAL DE EDUCACION ENRIQUE GUZM...  GOBIERNO NACIONAL
4  MUNICIPALIDAD PROVINCIAL DE YAROWILCA - CHAVIN...  GOBIERNO LOCAL

  entidad_departamento  codigoconvocatoria  objetocontractual  \
0                LIMA          497738          Bien
1                PUNO          486662          Bien
2                LORETO          458421          Bien
3                LIMA          475997          Bien
4            HUANUCO          457798  Consultoría de Obra

  tipoprocesoseleccion  proceso  ...  \
0  Contratación Internacional  INTER-PROC-12-2018-UNMSM-1  ...
1  Comparación de Precios  COMPRE-SM-35-2018-OEC/GR PUNO-1  ...
2  Adjudicación Simplificada  AS-SM-1-2018-GRL-DRSL/30.01-1  ...
3  Adjudicación Simplificada  AS-SM-17-2018-UNE-1  ...
4  Adjudicación Simplificada  AS-SM-3-2018-MPY-1  ...

  monto_adjudicado_item_soles  departamento_item  moneda  \
0          46,799.15          LIMA  Dólar Norteamericano
1          39,968.50          PUNO          Soles
2        105,000.00        LORETO          Soles
3        130,784.00          LIMA          Soles
4        341,189.11        HUANUCO          Soles
```

Figura 5.2. Carga del CSV con pandas y validación inicial de estructura.

5.3. Fase 3: Realizar limpieza básica

En la tercera fase se realizó un laboratorio de limpieza básica. Primero se generó una copia de trabajo y se creó una versión sucia del dataset para practicar la detección de problemas de calidad. Posteriormente, se identificaron valores nulos y duplicados, y se generó un archivo limpio en data/processed.

En la ejecución documentada, el dataset sucio alcanzó 100,109 filas y 26 columnas, con 3,273 valores nulos. Luego de aplicar dropna y drop_duplicates, el dataset limpio quedó con 97,987

filas, 26 columnas, 0 valores nulos y 0 filas duplicadas. Esta fase permitió dejar constancia de un flujo reproducible de limpieza sin alterar el archivo original.

+

jupyter02_limpieza_basicaLast Checkpoint: 12 minutes ago

FileEditViewRunKernelSettingsHelp

Trusted

JupyterLabPython 3 (ipykernel)

```
    f"Se esperaban 20 nulos nuevos, pero se agregaron {nulos_agregados}."
)

assert duplicados_agregados == 20, (
    f"Se esperaban 20 duplicados nuevos, pero se detectaron "
    f"{duplicados_agregados}."
)

print("\nValidación superada:")
print("✓ Se agregaron exactamente 20 valores nulos.")
print("✓ Se agregaron exactamente 20 filas.")
print("✓ Se generaron exactamente 20 duplicados nuevos.")
print("✓ El archivo original permanece sin modificaciones.")
```

=====

INFORMACIÓN INICIAL DEL DATASET

=====

Cantidad de filas: 100009

Cantidad de columnas: 26

Dimensión original: (100009, 26)

Cantidad inicial de nulos: 3253

Cantidad inicial de duplicados: 0

Columnas disponibles:

['anio', 'codigoentidad', 'entidad_ruc', 'entidad', 'tipoentidad', 'entidad_departamento', 'codigoconvocatoria', 'objetocontractual', 'tipoprocesoseleccion', 'proceso', 'descripcion_proceso', 'n_item', 'descripcion_item', 'estado_item', 'cantidad_adjudicado_item', 'monto_referencial_item_soles', 'monto_adjudicado_item_soles', 'departamento_item', 'moneda', 'unidad_medida', 'ruc_proveedor', 'proveedor', 'tipo_proveedor', 'fecha_convocatoria', 'fecha_buenapro', 'fecha_consentimiento_bp']

=====

20 VALORES NULOS INSERTADOS

=====

[]:

jupyter02_limpieza_basicaLast Checkpoint: 13 minutes ago

FileEditViewRunKernelSettingsHelp

Trusted

JupyterLabPython 3 (ipykernel)

```
    f"Se esperaban 20 nulos nuevos, pero se agregaron {nulos_agregados}."
)

assert duplicados_agregados == 20, (
    f"Se esperaban 20 duplicados nuevos, pero se detectaron "
    f"{duplicados_agregados}."
)

print("\nValidación superada:")
print("✓ Se agregaron exactamente 20 valores nulos.")
print("✓ Se agregaron exactamente 20 filas.")
print("✓ Se generaron exactamente 20 duplicados nuevos.")
print("✓ El archivo original permanece sin modificaciones.")
```

=====

20 VALORES NULOS INSERTADOS

=====

Fila 0 | Columna: anio | Valor anterior: 2018 | Valor nuevo: NULL

Fila 1 | Columna: codigoentidad | Valor anterior: 49.0 | Valor nuevo: NULL

Fila 2 | Columna: entidad_ruc | Valor anterior: 20408454299.0 | Valor nuevo: NULL

Fila 3 | Columna: entidad | Valor anterior: UNIVERSIDAD NACIONAL DE EDUCACION ENRIQUE GUZMAN Y VALLE | Valor nuevo: NULL

Fila 4 | Columna: tipoentidad | Valor anterior: GOBIERNO LOCAL | Valor nuevo: NULL

Fila 5 | Columna: entidad_departamento | Valor anterior: LIMA | Valor nuevo: NULL

Fila 6 | Columna: codigoconvocatoria | Valor anterior: 484775 | Valor nuevo: NULL

Fila 7 | Columna: objetocontractual | Valor anterior: Servicio | Valor nuevo: NULL

Fila 8 | Columna: tipoprocesoseleccion | Valor anterior: Subasta Inversa Electrónica | Valor nuevo: NULL

Fila 9 | Columna: proceso | Valor anterior: AS-SM-5-2018-CS-OBRA-MDSJB-1 | Valor nuevo: NULL

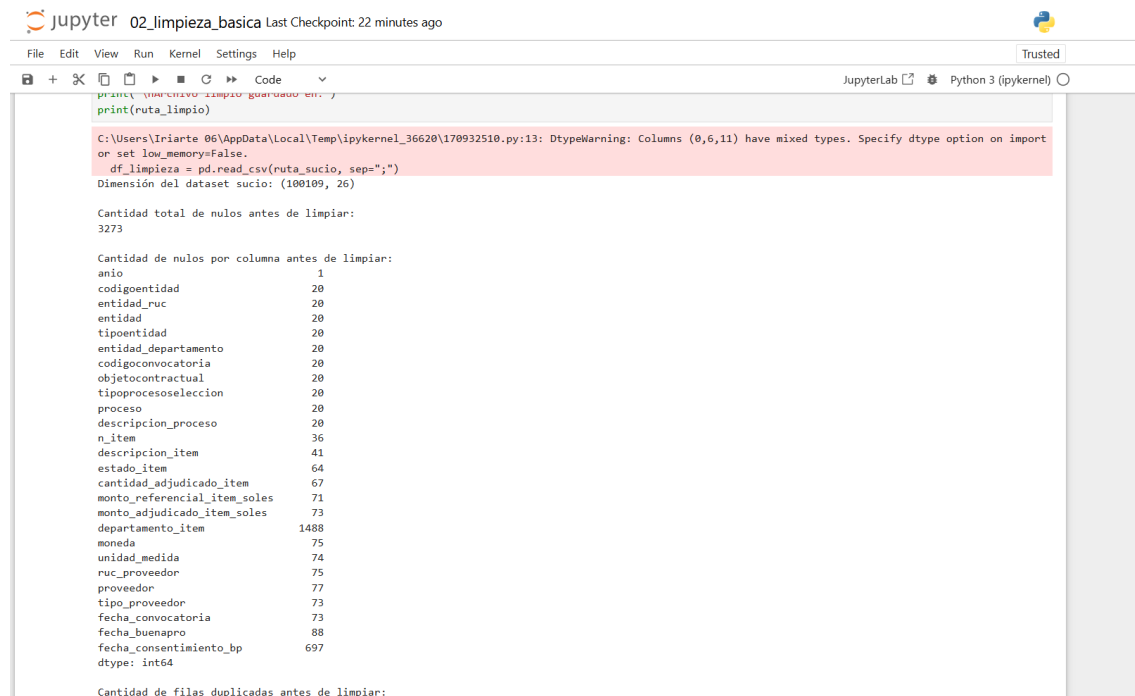
Fila 10 | Columna: descripcion_proceso | Valor anterior: Ejecución de la obra: "MEJORAMIENTO DEL SISTEMA DE RIEGO SISANJATO - CONCHAO - DISTRITO DE N

Fila 11 | Columna: n_item | Valor anterior: 1 | Valor nuevo: NULL

Fila 12 | Columna: descripcion_item | Valor anterior: CONTRATACIÓN DEL SERVICIO DE MENSAJERÍA Y PAQUETERIA A NIVEL LOCAL Y NACIONAL | Valor nuevo: NU

LL

[]:



The screenshot shows a JupyterLab window titled '02_limpieza_basica'. The code cell contains the following Python code:

```
print("Archivo limpio guardado en:")  
print(ruta_limpio)  
  
C:\Users\Triarte 06\AppData\Local\Temp\ipykernel_36620\170932510.py:13: DtypeWarning: Columns (0,6,11) have mixed types. Specify dtype option on import  
or set low_memory=False.  
df_limpieza = pd.read_csv(ruta_sucio, sep=";")  
Dimensión del dataset sucio: (100109, 26)  
  
Cantidad total de nulos antes de limpiar:  
3273  
  
Cantidad de nulos por columna antes de limpiar:  
anio 1  
codigoentidad 20  
entidad_ruc 20  
entidad 20  
tipoentidad 20  
entidad_departamento 20  
codigoconvocatoria 20  
objetocontractual 20  
tipoprocesoseleccion 20  
proceso 20  
descripcion_proceso 20  
n_item 36  
descripcion_item 41  
estado_item 64  
cantidad_adjudicado_item 67  
monto_referencial_item_soles 71  
monto_adjudicado_item_soles 73  
departamento_item 1488  
moneda 75  
unidad_medida 74  
ruc_proveedor 75  
proveedor 77  
tipo_proveedor 73  
fecha_convocatoria 73  
fecha_buenapro 88  
fecha_consentimiento_bp 697  
dtype: int64  
  
Cantidad de filas duplicadas antes de limpiar:
```

The output of the code is displayed below the code cell, showing the file path for the cleaned data and the dimensions of the dataset before cleaning.

Figura 5.3. Diagnóstico y limpieza de nulos y duplicados.

5.4. Fase 4: Creación de Features

La cuarta fase transformó la base limpia en una base modelable con nuevas variables o features. En esta fase se trabajó con la versión limpia utilizada para el modelamiento, que contenía 3,875 registros y 13 columnas base relacionadas con sanciones, proveedor, motivo de infracción, duración y variable objetivo.

Se crearon variables temporales a partir de la fecha de inicio de sanción, como ANIO_INICIO_SANCION, MES_INICIO_SANCION y TRIMESTRE_INICIO_SANCION. También se generaron variables de duración, como DURACION_MESES, DURACION_ANIOS, SANCION_MAYOR_1_ANIO y SANCION_MAYOR_3_ANIOS. Además,

se construyeron variables asociadas al tipo de RUC, motivo de infracción e historial del proveedor.

La base final featured quedó con 3,875 registros y 42 columnas. La distribución del target mostró 3,139 casos con valor 0 y 736 casos con valor 1, lo que confirma un desbalance de clases que debe ser considerado en el entrenamiento y evaluación del modelo.

```
03_feature_engineering Last Checkpoint: 12 hours ago
```

File Edit View Run Kernel Settings Help

Python 3 (ipykernel)

```
print("\nColumnas originales:")
print(df_features.columns.tolist())
print("\nPrimeras filas:")
display(df_features.head())
```

Dimensión del dataset limpio: (3875, 13)

Columnas originales:

['RUC', 'NOMBRE_RAZONODENOMINACIONSOCIAL', 'FECHA_INICIO', 'FECHA_FIN', 'NUMERO_RESOLUCION', 'ID_MOTIVO_INFRACCION', 'DE_MOTIVO_INFRACCION', 'motivo_grupo', 'duracion_dias', 'duracion_grupo', 'ruc_prefijo', 'fecha_segunda', 'target_sancionado_24m']

Primeras filas:

	RUC	NOMBRE_RAZONODENOMINACIONSOCIAL	FECHA_INICIO	FECHA_FIN	NUMERO_RESOLUCION	ID_MOTIVO_INFRACCION	DE_MOTIVO_INFRACCION
0	10000282125	GOMEZ CASTRO JUANA	20200616	20230716.0	1110-2020-TCE-S2	215,	Presentar documentos falsos o adulterados a la...
1	10000710623	OLIVEIRA DE MACHUCA LITA	20191107	20230207.0	2903-2019-TCE-S2	214,215,	Presentar información inexacta a las Entidades...
2	10001253609	AGUIRRE VARGAS GUSTAVO ALFREDO	20190807	20220907.0	2147-2019-TCE-S3	214,215,	Presentar información inexacta a las Entidades...
3	10002101756	MENDOZA PENA JULIO CESAR	20231005	20240205.0	3820-2023-TCE-S6	241,	f) Ocasionar que la Entidad Resuelva el contra...
4	10002381449	ROSALES ANCAJIMA MIRELLA NANCY	20200617	20230617.0	1126-2020-TCE-S4	245,	j) Presentar documentos falsos o adulterados a...

```
[5]: # =====
# 6. CREAR FEATURES TEMPORALES
# =====

df_features["ANIO_INICIO_SANCION"] = df_features["FECHA_INICIO_DT"].dt.year
df_features["MES_INICIO_SANCION"] = df_features["FECHA_INICIO_DT"].dt.month
df_features["TRIMESTRE_INICIO_SANCION"] = df_features["FECHA_INICIO_DT"].dt.quarter

df_features["PERIODO_INICIO_SANCION"] = (
    df_features["ANIO_INICIO_SANCION"].astype("Int64").astype(str)
    + "-"
    + df_features["MES_INICIO_SANCION"].astype("Int64").astype(str).str.zfill(2)
)

print("Features temporales creadas:")
display(df_features[[
    "FECHA_INICIO_DT",
    "ANIO_INICIO_SANCION",
    "MES_INICIO_SANCION",
    "TRIMESTRE_INICIO_SANCION",
    "PERIODO_INICIO_SANCION"
]].head())
```

Features temporales creadas:

	FECHA_INICIO_DT	ANIO_INICIO_SANCION	MES_INICIO_SANCION	TRIMESTRE_INICIO_SANCION	PERIODO_INICIO_SANCION
0	2020-06-16	2020	6	2	2020-06
1	2019-11-07	2019	11	4	2019-11
2	2019-08-07	2019	8	3	2019-08
3	2023-10-05	2023	10	4	2023-10
4	2020-06-17	2020	6	2	2020-06

```
[6]: # =====
```

```
# 7. CREAR FEATURES DE DURACIÓN
# =====

df_features["DURACION_MESES"] = df_features["duracion_dias"] / 30
df_features["DURACION_ANIOS"] = df_features["duracion_dias"] / 365

df_features["SANCION_MAYOR_1_ANIO"] = np.where(
    df_features["duracion_dias"] > 365,
    1,
    0
)

df_features["SANCION_MAYOR_3_ANIOS"] = np.where(
    df_features["duracion_dias"] > 1095,
    1,
    0
)

# Corrección para evitar advertencias en el logaritmo
df_features["DURACION_DIAS_CORREGIDA"] = df_features["duracion_dias"].where(
    df_features["duracion_dias"] >= 0,
    np.nan
)

df_features["LOG_DURACION_DIAS"] = np.log1p(df_features["DURACION_DIAS_CORREGIDA"])

print("Features de duración creadas:")
display(df_features[
    "duracion_dias",
    "DURACION_DIAS_CORREGIDA",
    "DURACION_MESES",
    "DURACION_ANIOS",
    "SANCION_MAYOR_1_ANIO",
    "SANCION_MAYOR_3_ANIOS",
    "LOG_DURACION_DIAS"
].head())
```

Features de duración creadas:

	duracion_dias	DURACION_DIAS_CORREGIDA	DURACION_MESES	DURACION_ANIOS	SANCION_MAYOR_1_ANIO	SANCION_MAYOR_3_ANIOS	LOG_DURACION_DIAS
0	1125.0	1125.0	37.500000	3.082192	1	1	7.026427
1	1188.0	1188.0	39.600000	3.254795	1	1	7.080868
2	1127.0	1127.0	37.566667	3.087671	1	1	7.028201
3	123.0	123.0	4.100000	0.336986	0	0	4.820282
4	1095.0	1095.0	36.500000	3.000000	1	0	6.999422

```
[9]: # =====
# 8. CREAR FEATURES DEL RUC
# =====

df_features["RUC_STR"] = df_features["RUC"].astype(str).str.zfill(11)
df_features["RUC_PREFIJO"] = df_features["RUC_STR"].str[:2]

df_features["ES_PERSONA_NATURAL"] = np.where(
    df_features["RUC_PREFIJO"] == "10",
    1,
    0
)

df_features["ES_PERSONA_JURIDICA"] = np.where(
    df_features["RUC_PREFIJO"] == "20",
    1,
    0
)

df_features["ES_OTRO_TIPO_RUC"] = np.where(
    ~df_features["RUC_PREFIJO"].isin(["10", "20"]),
    1,
    0
)

print("Features de RUC creadas:")
display(df_features[[
    "RUC",
    "RUC_STR",
    "RUC_PREFIJO",
    "ES_PERSONA_NATURAL",
    "ES_PERSONA_JURIDICA",
    "ES_OTRO_TIPO_RUC"
]].head())
```

Features de RUC creadas:

	RUC	RUC_STR	RUC_PREFIJO	ES_PERSONA_NATURAL	ES_PERSONA_JURIDICA	ES_OTRO_TIPO_RUC
0	10000282125	10000282125	10	1	0	0
1	10000710623	10000710623	10	1	0	0
2	10001253609	10001253609	10	1	0	0
3	10002101756	10002101756	10	1	0	0
4	10002381449	10002381449	10	1	0	0

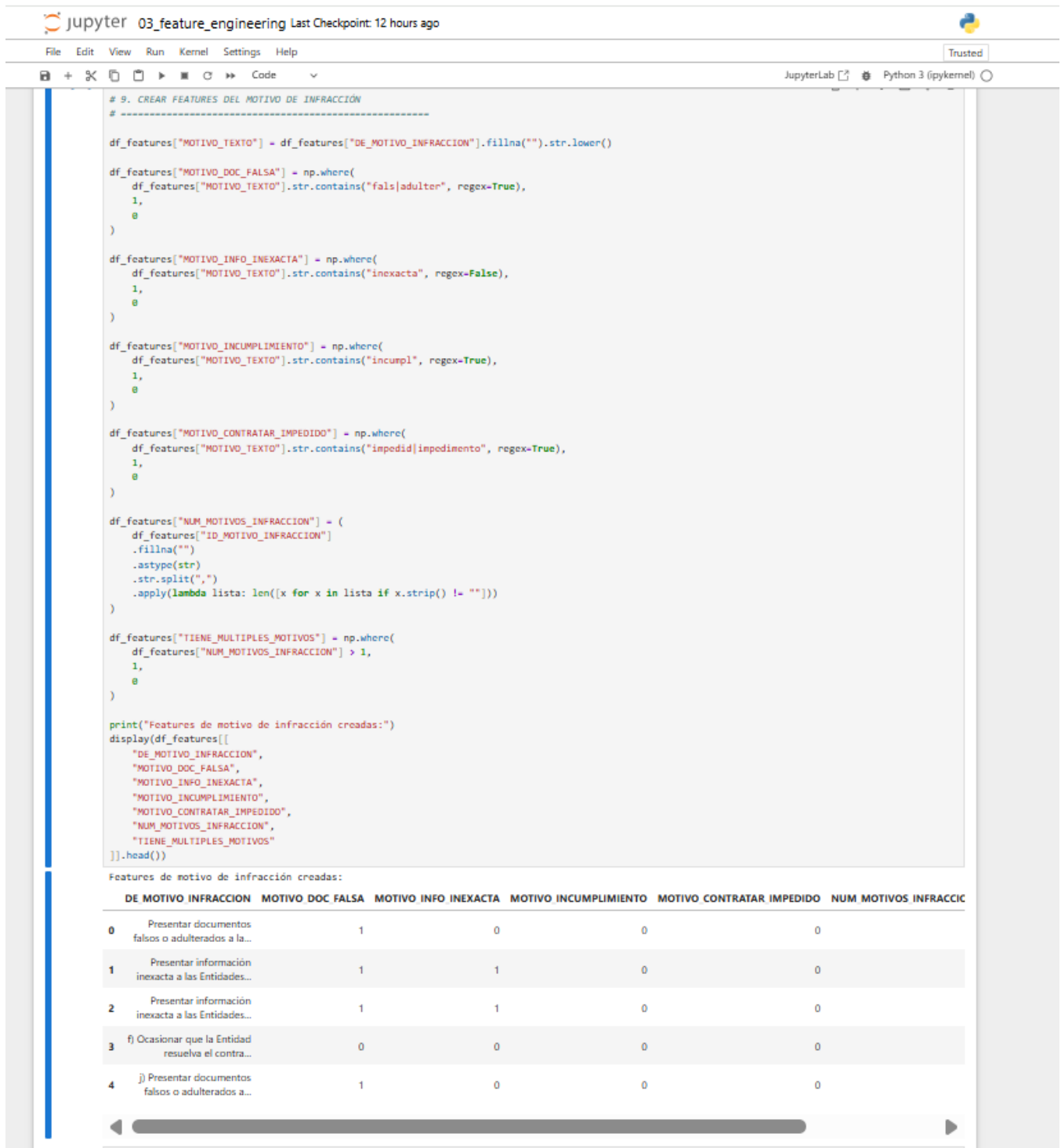


Figura 5.4. Creación de features y distribución de la variable objetivo.

5.5. Fase 5: Entrenamiento del modelo

La quinta fase consistió en entrenar un primer modelo de clasificación. Se separaron las variables predictoras x y la variable objetivo y . Luego, las variables categóricas fueron convertidas a variables numéricas mediante `get_dummies`, lo que permitió que el algoritmo pudiera procesarlas.

La división del dataset se hizo en entrenamiento y prueba con una proporción 80/20. Se utilizó `stratify=y` para conservar la distribución de clases en ambos subconjuntos. Como modelo base se entrenó una Regresión Logística con `max_iter=20000` y `class_weight="balanced"`, con el propósito de reducir el sesgo frente a la clase mayoritaria.

El modelo base obtuvo un `accuracy` inicial de 0.7135. Para la clase positiva 1, que representa el evento de riesgo, la precisión fue 0.33, el `recall` 0.48 y el F1-score 0.39. Estos resultados sirvieron como primera referencia de desempeño para la evaluación posterior.

```
[1]: # =====
# FASE 5
# ENTRENAR MODELO SIMPLE DE CLASIFICACIÓN
# RISK SCORE OSCE
# =====

import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# =====
# 1. Definir ruta del archivo de entrada
# =====

ruta_featured = r"C:\Proyecto-Risk-Score-OECE\data\processed\CONOSCE_ADJUDICACIONES_2018_2026_CONSOLIDADO_FEATURED.csv"

# =====
# 2. Cargar el dataset con features
# IMPORTANTE: el archivo usa punto y coma como separador
# =====

df_featured = pd.read_csv(ruta_featured, sep=";")

# =====
# 3. Crear una copia de trabajo
# =====

df_modelo = df_featured.copy()

# =====
# 4. Mostrar información inicial
# =====

print("Dimensión del dataset featured:", df_modelo.shape)

print("\nColumnas disponibles:")
print(df_modelo.columns.tolist())

print("\nPrimeras filas:")
display(df_modelo.head())
```

Dimensión del dataset featured: (3875, 42)

Columnas disponibles:

```
['RUC', 'NOMBRE_RAZONODENOMINACIONESOCIAL', 'FECHA_INICIO', 'FECHA_FIN', 'NUMERO_RESOLUCION', 'ID_MOTIVO_INFRACCION', 'DE_MOTIVO_INFRACCION', 'motivo_grupo', 'duracion_dias', 'duracion_grupo', 'ruc_prefijo', 'fecha_segunda', 'target_sancionado_24m', 'FECHA_INICIO_DT', 'FECHA_FIN_DT', 'ANIO_INICIO_SANCCION', 'MES_INICIO_SANCCION', 'TRIMESTRE_INICIO_SANCCION', 'PERIODO_INICIO_SANCCION', 'DURACION_MESES', 'DURACION_ANIOS', 'SANCION_MAYOR_1_ANIO', 'SANCION_MAYOR_3_ANIOS', 'LOG_DURACION_DIAS', 'DURACION_DIAS_CORREGIDA', 'RUC_STR', 'RUC_PREFIJO', 'ES_PERSONA_NATURAL', 'ES_PERSONA_JURIDICA', 'ES_OTRO_TIPO_RUC', 'MOTIVO_TEXTO', 'MOTIVO_DOC_FALSA', 'MOTIVO_INFO_INEXACTA', 'MOTIVO_INCUMPLIMIENTO', 'MOTIVO_CONTRATAR_IMPEDIDO', 'NUM_MOTIVOS_INFRACCION', 'TIENE_MULTIPLES_MOTIVOS', 'TOTAL_SANCIONES_PROVEEDOR', 'ORDEN_SANCCION_PROVEEDOR', 'PROVEEDOR_CON_MULTIPLES_SANCIONES', 'FECHA_PRIMERA_SANCCION_PROVEEDOR', 'DIAS_DESDE_PRIMERA_SANCCION']
```

Primeras filas:

K, DIAS_DESDE_PRIMERA_SANCCION]

Primeras filas:

	RUC	NOMBRE_RAZON	NODENOMINACION	SOCIAL	FECHA_INICIO	FECHA_FIN	NUMERO_RESOLUCION	ID_MOTIVO_INFRACCION	DE_MOTIVO_INFRACCION
0	2029124996	G & D CORPORACION DE NEGOCIOS LACTEOS S.A. COR...			20050608	NaN	508-2005-TC-SU	8	DOCUMENTOS FALSOS
1	10000282125	GOMEZ CASTRO JUANA			20200616	20230716.0	1110-2020-TCE-S2	215,	Presentar documentos falsos o adulterados a la...
2	10000710623	OLIVEIRA DE MACHUCA LITA			20191107	20230207.0	2903-2019-TCE-S2	214,215,	Presentar información inexacta a las Entidades...
3	10001253609	AGUIRRE VARGAS GUSTAVO ALFREDO			20190807	20220907.0	2147-2019-TCE-S3	214,215,	Presentar información inexacta a las Entidades...
4	10002101756	MENDOZA PENA JULIO CESAR			20231005	20240205.0	3820-2023-TCE-S6	241,	f) Ocasionar que la Entidad resuelva el contra... Re

5 rows x 42 columns

```
[2]: # =====
# 5. Definir variable objetivo
# =====

target = "target_sancionado_24m"

print("variable objetivo:", target)

print("\nDistribución de la variable objetivo:")
print(df_modelo[target].value_counts())

print("\nDistribución porcentual:")
print(df_modelo[target].value_counts(normalize=True) * 100)

variable objetivo: target_sancionado_24m

Distribución de la variable objetivo:
target_sancionado_24m
0    3139
1     736
Name: count, dtype: int64

Distribución porcentual:
target_sancionado_24m
0    81.006452
1    18.993548
Name: proportion, dtype: float64
```



```
[3]: # =====
# 6. Seleccionar variables predictoras iniciales
# =====

columnas_modelo = [
    # Variables temporales creadas en Fase 4
    "ANIO_INICIO_SANCION",
    "MES_INICIO_SANCION",
    "TRIMESTRE_INICIO_SANCION",

    # Variables de duración / severidad
    "DURACION_MESES",
    "DURACION_ANIOS",
    "SANCION_MAYOR_1_ANIO",
    "SANCION_MAYOR_3_ANIOS",
    "LOG_DURACION_DIAS",

    # Variables del tipo de proveedor
    "ES_PERSONA_NATURAL",
    "ES_PERSONA_JURIDICA",
    "ES_OTRO_TIPO_RUC",

    # Variables del motivo de infracción
    "MOTIVO_DOC_FALSA",
    "MOTIVO_INFO_INEXACTA",
    "MOTIVO_INCUMPLIMIENTO",
    "MOTIVO_CONTRATAR_IMPEDIDO",
    "NUM_MOTIVOS_INFRACCION",
    "TIENE_MULTIPLES_MOTIVOS",

    # Variables categóricas originales o agrupadas
    "motivo_grupo",
    "duracion_grupo"
]

# Verificar que las columnas existan
columnas_existentes = [col for col in columnas_modelo if col in df_modelo.columns]
columnas_faltantes = [col for col in columnas_modelo if col not in df_modelo.columns]

print("Columnas que sí existen y se usarán:")
print(columnas_existentes)

print("\nColumnas faltantes, no se usarán:")
print(columnas_faltantes)

# Quedarnos solo con columnas existentes + target
df_modelo = df_modelo[columnas_existentes + [target]].copy()

print("\nDimensión del dataset para modelamiento:")
print(df_modelo.shape)

display(df_modelo.head())

Columnas que sí existen y se usarán:
['ANIO_INICIO_SANCION', 'MES_INICIO_SANCION', 'TRIMESTRE_INICIO_SANCION', 'DURACION_MESES', 'DURACION_ANIOS', 'SANCION_MAYOR_1_ANIO', 'SANCION_MAYOR_3_ANIOS', 'LOG_DURACION_DIAS', 'ES_PERSONA_NATURAL', 'ES_PERSONA_JURIDICA', 'ES_OTRO_TIPO_RUC', 'MOTIVO_DOC_FALSA', 'MOTIVO_INFO_INEXACTA', 'MOTIVO_INCUMPLIMIENTO', 'MOTIVO_CONTRATAR_IMPEDIDO', 'NUM_MOTIVOS_INFRACCION', 'TIENE_MULTIPLES_MOTIVOS', 'motivo_grupo', 'duracion_grupo']

Columnas faltantes, no se usarán:
--
```

Columnas tratantes, no se usaran:
[]

Dimensión del dataset para modelamiento:
(3875, 20)

	ANIO_INICIO_SANCION	MES_INICIO_SANCION	TRIMESTRE_INICIO_SANCION	DURACION_MESES	DURACION_ANIOS	SANCION_MAYOR_1_ANIO	SANCION_MAYOR_3
0	2005	6	2	NaN	NaN	0	
1	2020	6	2	37.500000	3.082192	1	
2	2019	11	4	39.600000	3.254795	1	
3	2019	8	3	37.566667	3.087671	1	
4	2023	10	4	4.100000	0.336986	0	

```
[4]: # =====
# 7. Limpieza básica antes del modelamiento
# =====

# Reemplazar infinitos por NaN
df_modelo = df_modelo.replace([np.inf, -np.inf], np.nan)

# Mostrar cantidad de nulos por columna
print("Nulos antes del tratamiento:")
print(df_modelo.isnull().sum())

# Separar columnas numéricas y categóricas
columnas_numericas = df_modelo.select_dtypes(include=["int64", "float64"]).columns.tolist()
columnas_categoricas = df_modelo.select_dtypes(include=["object"]).columns.tolist()

# Quitar target de las numéricas si aparece allí
if target in columnas_numericas:
    columnas_numericas.remove(target)

# Rellenar nulos numéricos con la mediana
for col in columnas_numericas:
    df_modelo[col] = df_modelo[col].fillna(df_modelo[col].median())

# Rellenar nulos categóricos con "SIN_DATO"
for col in columnas_categoricas:
    df_modelo[col] = df_modelo[col].fillna("SIN_DATO")

print("\nNulos después del tratamiento:")
print(df_modelo.isnull().sum())
```

Nulos antes del tratamiento:
ANIO_INICIO_SANCION 0

```
df_modelo[CO2] = df_modelo[CO2].fillna("sin_otto")
```

```
print("\nNulos después del tratamiento:")  
print(df_modelo.isnull().sum())
```

Nulos antes del tratamiento:

ANIO_INICIO_SANCION	0
MES_INICIO_SANCION	0
TRIMESTRE_INICIO_SANCION	0
DURACION_MESES	528
DURACION_ANIOS	528
SANCION_MAYOR_1_ANIO	0
SANCION_MAYOR_3_ANIOS	0
LOG_DURACION_DIAS	539
ES_PERSONA_NATURAL	0
ES_PERSONA_JURIDICA	0
ES_OTRO_TIPO_RUC	0
MOTIVO_DOC_FALSA	0
MOTIVO_INFO_INEXACTA	0
MOTIVO_INCUMPLIMIENTO	0
MOTIVO_CONTRATAR_IMPEDIDO	0
NUM_MOTIVOS_INFRACCION	0
TIENE_MULTIPLES_MOTIVOS	0
motivo_grupo	0
duracion_grupo	0
target_sancionado_24m	0
dtype: int64	

Nulos después del tratamiento:

ANIO_INICIO_SANCION	0
MES_INICIO_SANCION	0
TRIMESTRE_INICIO_SANCION	0
DURACION_MESES	0
DURACION_ANIOS	0
SANCION_MAYOR_1_ANIO	0
SANCION_MAYOR_3_ANIOS	0
LOG_DURACION_DIAS	0
ES_PERSONA_NATURAL	0
ES_PERSONA_JURIDICA	0
ES_OTRO_TIPO_RUC	0
MOTIVO_DOC_FALSA	0
MOTIVO_INFO_INEXACTA	0
MOTIVO_INCUMPLIMIENTO	0
MOTIVO_CONTRATAR_IMPEDIDO	0
NUM_MOTIVOS_INFRACCION	0
TIENE_MULTIPLES_MOTIVOS	0
motivo_grupo	0
duracion_grupo	0
target_sancionado_24m	0
dtype: int64	

```
[9]: class_weight="balanced"
```

```
[10]: # =====
# 13. Generar predicciones iniciales
# =====

y_pred = modelo.predict(X_test)

# Probabilidad de pertenecer a la clase positiva: 1
y_prob = modelo.predict_proba(X_test)[:, 1]

print("Primeras 10 predicciones:")
print(y_pred[:10])

print("\nPrimeras 10 probabilidades de sanción futura:")
print(y_prob[:10])

Primeras 10 predicciones:
[0 1 0 0 1 0 1 1 0 1]

Primeras 10 probabilidades de sanción futura:
[0.4480394  0.63929793 0.41292219 0.45208491 0.60737889 0.38812931
 0.5796798  0.52096224 0.41661818 0.53530039]
```

```
[11]: # =====
# 14. Resultados iniciales del modelo
# =====

print("Accuracy inicial:")
print(accuracy_score(y_test, y_pred))

print("\nMatriz de confusión:")
print(confusion_matrix(y_test, y_pred))

print("\nReporte de clasificación:")
print(classification_report(y_test, y_pred))

Accuracy inicial:
0.7135483870967742

Matriz de confusión:
[[483 145]
 [ 77  70]]

Reporte de clasificación:
      precision    recall  f1-score   support

     0       0.86       0.77       0.81         628
     1       0.33       0.48       0.39         147

   accuracy                   0.71         775
  macro avg       0.59       0.62       0.60         775
 weighted avg       0.76       0.71       0.73         775
```

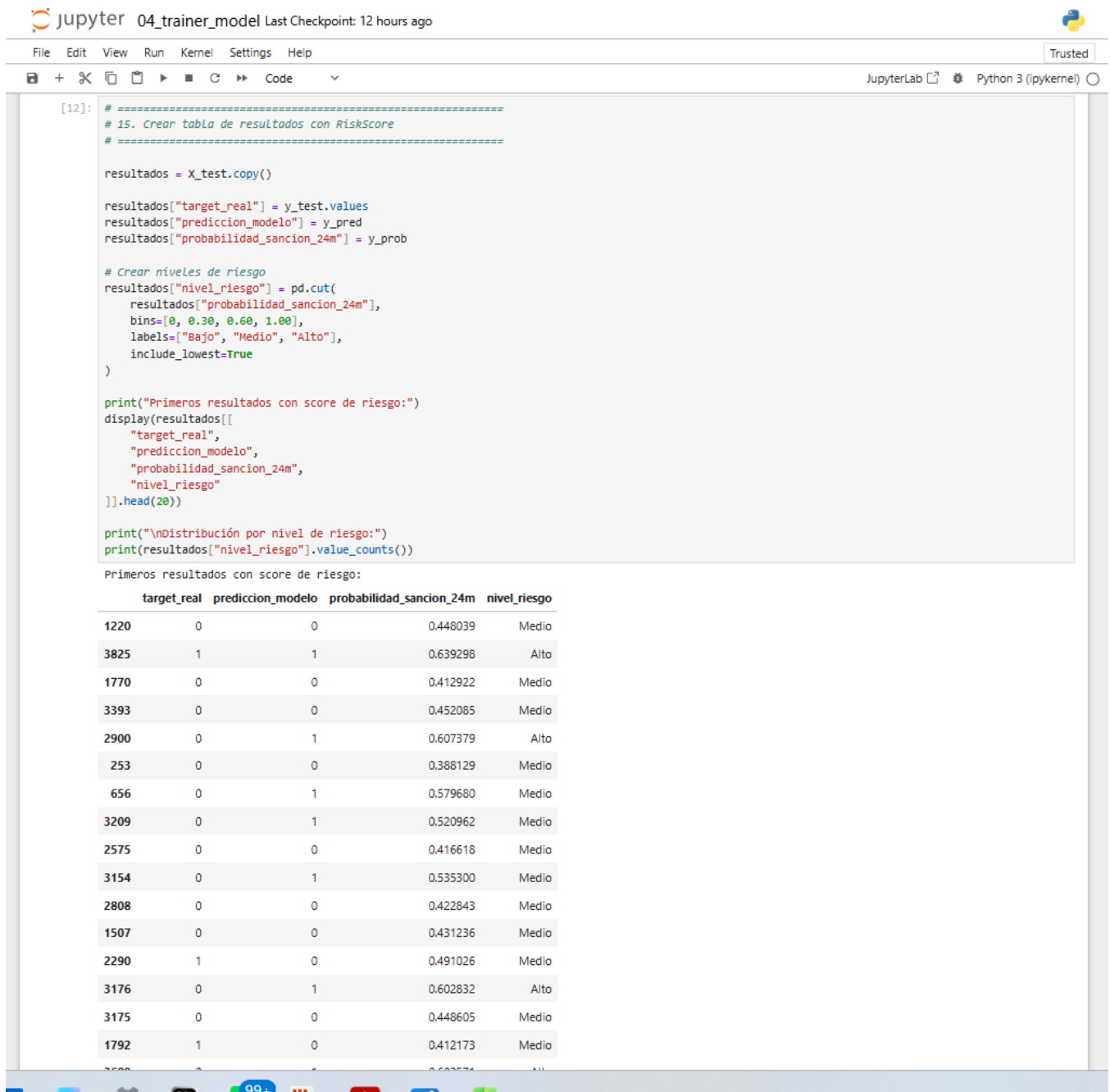


Figura 5.5. Entrenamiento del modelo base con Regresión Logística balanceada.

5.6. Fase 6: Evaluación del modelo

La sexta fase evaluó formalmente el desempeño del modelo usando el conjunto de prueba.

La evaluación se realizó mediante accuracy, matriz de confusión, classification report,

precision, recall, F1-score y ROC-AUC. En este proyecto, la métrica clave es el desempeño sobre la clase 1, porque representa proveedores con sanción posterior dentro de 24 meses.

Además de evaluar la Regresión Logística, se entrenó y evaluó un Random Forest balanceado. La comparación mostró que la Regresión Logística obtuvo un accuracy de 0.7135 y un recall de clase 1 de 0.4762, mientras que el Random Forest balanceado obtuvo un accuracy de 0.7071 y un recall de clase 1 de 0.3946. Esto muestra que ambos modelos tuvieron desempeño cercano, aunque con diferencias en la sensibilidad frente a la clase de riesgo.

También se incorporó un análisis de importancia de variables para el Random Forest. Las variables con mayor importancia fueron MES_INICIO_SANCION, ANIO_INICIO_SANCION, DURACION_ANIOS, LOG_DURACION_DIAS y DURACION_MESES. Esto sugiere que el componente temporal y la duración de las sanciones son señales relevantes para el modelo.

```
jupyter 05_evaluator_model Last Checkpoint: 11 hours ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

print(f"Accuracy del modelo: {accuracy:.4f}")

print("\nMatriz de confusión:")
print(matriz_confusion)

print("\nReporte de clasificación:")
print(reporte_clasificacion)

print("\nMétricas específicas para la clase positiva 1:")
print(f"Precision clase 1: {precision_clase_1:.4f}")
print(f"Recall clase 1: {recall_clase_1:.4f}")
print(f"F1-score clase 1: {f1_clase_1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")

=====
RESULTADOS DE EVALUACIÓN - RISK SCORE OSCE
=====

Accuracy del modelo: 0.7135

Matriz de confusión:
[[483 145]
 [ 77  70]]

Reporte de clasificación:
      precision    recall  f1-score   support

     0       0.86       0.77       0.81       628
     1       0.33       0.48       0.39       147

 accuracy          0.71       0.71       0.71       775
 macro avg         0.59       0.62       0.60       775
weighted avg         0.76       0.71       0.73       775

Métricas específicas para la clase positiva 1:
Precision clase 1: 0.3256
Recall clase 1:    0.4762
F1-score clase 1: 0.3867
ROC-AUC:          0.6577

[7]: # =====
```

```
print("\nMétricas específicas para clase positiva 1:")
print(f"Precision clase 1 RF: {precision_rf_1:.4f}")
print(f"Recall clase 1 RF: {recall_rf_1:.4f}")
print(f"F1-score clase 1 RF: {f1_rf_1:.4f}")
print(f"ROC-AUC RF: {roc_auc_rf:.4f}")
```

```
=====
RESULTADOS RANDOM FOREST BALANCEADO - RISK SCORE OSCE
=====
```

Accuracy Random Forest: 0.7071

Matriz de confusión:

```
[[490 138]
 [ 89  58]]
```

Reporte de clasificación:

	precision	recall	f1-score	support
0	0.85	0.78	0.81	628
1	0.30	0.39	0.34	147
accuracy			0.71	775
macro avg	0.57	0.59	0.58	775
weighted avg	0.74	0.71	0.72	775

Métricas específicas para clase positiva 1:

```
Precision clase 1 RF: 0.2959
Recall clase 1 RF: 0.3946
F1-score clase 1 RF: 0.3382
ROC-AUC RF: 0.6579
```

```
[10]: # -----
```

Figura 5.6. Comparación de modelos mediante métricas de clasificación.

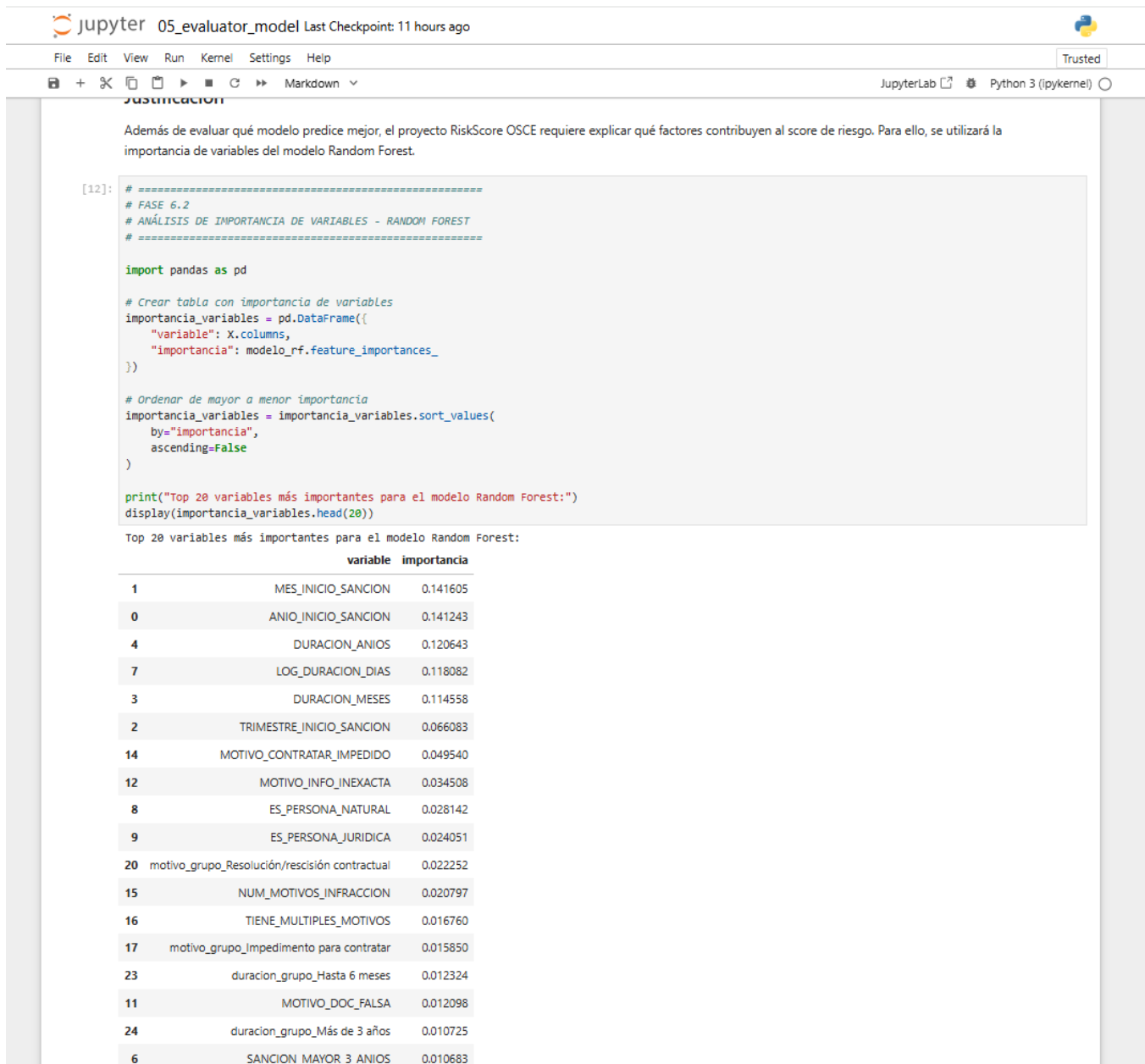


Figura 5.7. Importancia de variables en el modelo Random Forest.

5.7. Fase 7: Guardar el Pipeline completo en .pkl & .joblib

La séptima fase consistió en guardar el modelo final para que pueda ser reutilizado posteriormente sin volver a entrenarlo. Para ello, el notebook reconstruyó de forma autosuficiente el flujo mínimo: carga del dataset featured, selección de variables, limpieza

básica, codificación con `get_dummies`, división estratificada, entrenamiento del Random Forest balanceado y evaluación previa al guardado.

El modelo final fue guardado en formato `.joblib` y en formato `.pkl`. Además, se generó un artefacto completo en `.joblib` con metadatos del proyecto, variable objetivo, columnas usadas en entrenamiento, columnas numéricas y categóricas, medianas de imputación, parámetros del modelo, clases detectadas y métricas de validación.

La verificación posterior confirmó que tanto el archivo `.joblib` como el archivo `.pkl` pueden cargarse nuevamente y reproducir el mismo comportamiento predictivo. El artefacto cargado reportó el nombre del proyecto RiskScore OSCE, el target `target_sancionado_24m`, las clases `[0, 1]`, 26 columnas de entrenamiento y las métricas de validación del modelo.

```
jupyter 06_save_pipeline Last Checkpoint: 11 hours ago
File Edit View Run Kernel Settings Help Trusted
+ ✂ 📄 ▶ ⏸ ⏪ ⏩ ⏹ Markdown
JupyterLab Python 3 (ipykernel)
```

```
print(f"Precision clase 1: {precision_1:.4f}")
print(f"Recall clase 1: {recall_1:.4f}")
print(f"F1-score clase 1: {f1_1:.4f}")
print(f"ROC-AUC: {roc_auc:.4f}")

=====
RESULTADOS DEL MODELO FINAL ANTES DEL GUARDADO
=====

Accuracy del modelo final: 0.7071

Matriz de confusión:
[[490 138]
 [ 89  58]]

Reporte de clasificación:
      precision    recall  f1-score   support

     0       0.85       0.78       0.81        628
     1       0.30       0.39       0.34        147

   accuracy          0.71        775
  macro avg       0.57       0.59       0.58        775
 weighted avg       0.74       0.71       0.72        775

Métricas clase positiva 1:
Precision clase 1: 0.2959
Recall clase 1: 0.3946
F1-score clase 1: 0.3382
ROC-AUC: 0.6579
```

```
"columnas_numericas": columnas_numericas,
"columnas_categoricas": columnas_categoricas,
"medianas_numericas": medianas_numericas,
"parametros_modelo": modelo_final.get_params(),
"clases_modelo": modelo_final.classes_.tolist(),
"metricas_validacion": {
    "accuracy": accuracy,
    "precision_clase_1": precision_1,
    "recall_clase_1": recall_1,
    "f1_clase_1": f1_1,
    "roc_auc": roc_auc
}
}

joblib.dump(artefacto_final, ruta_artefacto, compress=3)

print("\nArchivos guardados correctamente:")
print("- Modelo .joblib:", ruta_joblib)
print("- Modelo .pkl:", ruta_pkl)
print("- Artefacto completo:", ruta_artefacto)
print("- Columnas de entrenamiento:", ruta_columnas)
```

```
Archivos guardados correctamente:
- Modelo .joblib: C:\Proyecto-Risk-Score-OECE\models\riskscore_osce_random_forest.joblib
- Modelo .pkl: C:\Proyecto-Risk-Score-OECE\models\riskscore_osce_random_forest.pkl
- Artefacto completo: C:\Proyecto-Risk-Score-OECE\models\artefacto_riskscore_osce_random_forest.joblib
- Columnas de entrenamiento: C:\Proyecto-Risk-Score-OECE\models\columnas_entrenamiento_riskscore_osce.csv
```

[10]: # -----

```
print("="*60)

print(f"\nAccuracy de verificación (.joblib): {accuracy_joblib:.4f}")

print("\nMatriz de confusión (.joblib):")
print(matriz_joblib)

print("\nReporte de clasificación (.joblib):")
print(reporte_joblib)
```

```
=====
VERIFICACIÓN DEL MODELO CARGADO DESDE .JOBLIB
=====
```

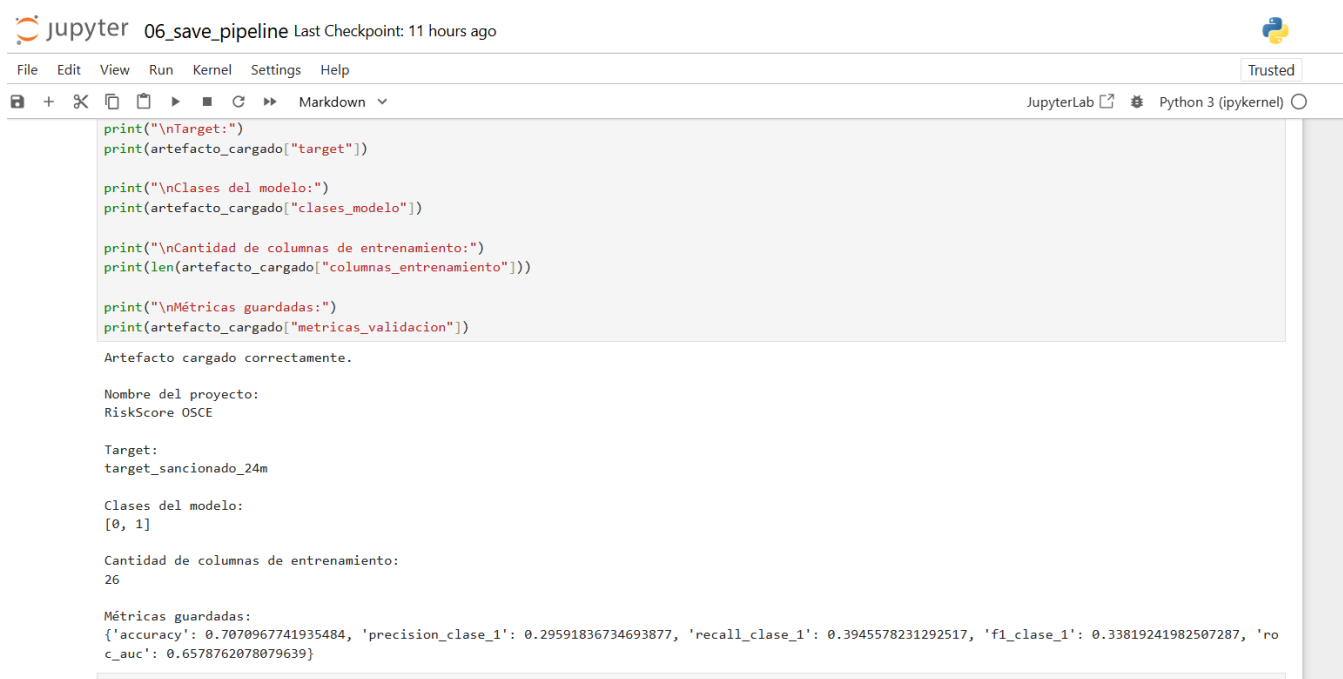
```
Accuracy de verificación (.joblib): 0.7071
```

```
Matriz de confusión (.joblib):
[[490 138]
 [ 89  58]]
```

```
Reporte de clasificación (.joblib):
```

	precision	recall	f1-score	support
0	0.85	0.78	0.81	628
1	0.30	0.39	0.34	147
accuracy			0.71	775
macro avg	0.57	0.59	0.58	775
weighted avg	0.74	0.71	0.72	775

[11]: # -----



```
print("\nTarget:")
print(artefacto_cargado["target"])

print("\nClases del modelo:")
print(artefacto_cargado["clases_modelo"])

print("\nCantidad de columnas de entrenamiento:")
print(len(artefacto_cargado["columnas_entrenamiento"]))

print("\nMétricas guardadas:")
print(artefacto_cargado["metricas_validacion"])

Artefacto cargado correctamente.

Nombre del proyecto:
RiskScore OSCE

Target:
target_sancionado_24m

Clases del modelo:
[0, 1]

Cantidad de columnas de entrenamiento:
26

Métricas guardadas:
{'accuracy': 0.7070967741935484, 'precision_clase_1': 0.29591836734693877, 'recall_clase_1': 0.3945578231292517, 'f1_clase_1': 0.33819241982507287, 'roc_auc': 0.6578762078079639}
```

Figura 5.8. Guardado del pipeline, artefacto completo y verificación de persistencia.

Conclusión del capítulo

El desarrollo del proyecto RiskScore OECE permitió implementar de manera integral un flujo reproducible de ciencia de datos orientado a la estimación del riesgo de sanción futura de proveedores adjudicados en contrataciones públicas. A partir de la integración, depuración y transformación de datos provenientes de fuentes oficiales, fue posible construir una base modelable, diseñar variables predictoras relevantes y entrenar modelos de clasificación binaria capaces de identificar patrones asociados al riesgo regulatorio.

El modelo constituye una prueba de concepto funcional que demuestra la viabilidad de aplicar técnicas de inteligencia artificial para fortalecer la gestión preventiva del riesgo. En este sentido, el proyecto cumple los objetivos académicos planteados y además proporciona una base técnica escalable para futuras optimizaciones, incorporación de nuevas variables y despliegue en entornos de apoyo a la toma de decisiones dentro de entidades públicas y organizaciones que interactúan con proveedores del Estado.

ANEXOS

Anexo 1: Problema de negocio y KPI alineado

Elemento	Desarrollo
Problema de negocio	Las entidades públicas, bancos, aseguradoras, consultoras de compliance y empresas privadas que contratan con proveedores del Estado necesitan anticipar riesgos asociados a proveedores que podrían ser sancionados posteriormente. Actualmente, la revisión suele ser reactiva y se centra en verificar sanciones existentes, pero no estima el riesgo futuro.
Objetivo del proyecto de IA	Desarrollar un modelo de machine learning de clasificación binaria que estime la probabilidad de que un proveedor adjudicado sea sancionado dentro de los 24 meses posteriores a la fecha de buena pro.
Variable objetivo	SANCIONADO_24M: 1 si el proveedor recibió al menos una sanción posterior dentro de los 24 meses siguientes a la buena pro; 0 si no registra sanción posterior dentro de dicho periodo.
Usuario potencial	Entidades públicas, áreas de abastecimiento, bancos, aseguradoras, empresas que realizan due diligence de proveedores, consultoras legales y áreas de cumplimiento.
Decisión que apoya	Priorización de revisiones, clasificación de proveedores por riesgo, solicitud de garantías adicionales, fortalecimiento del due diligence y asignación eficiente de recursos de control.

Elemento	Desarrollo
KPI de negocio principal	Tasa de detección de proveedores posteriormente sancionados.
KPI complementario 1	Porcentaje de sancionados futuros capturados dentro del top 10 % de mayor riesgo.
KPI complementario 2	Reducción del universo de revisión manual mediante priorización de casos de alto riesgo.
KPI complementario 3	Monto adjudicado expuesto a proveedores clasificados como alto riesgo.
KPI técnico asociado	Recall, precision, F1-score, ROC-AUC y PR-AUC.
Impacto esperado	Mejor prevención del riesgo regulatorio, asignación más eficiente de recursos de supervisión y reducción de exposición frente a proveedores con señales históricas de mayor riesgo.

Anexo 2: Datos necesarios, fuentes, volumetría y frecuencia

Tipo de dato	Fuente	Periodo	Uso dentro del proyecto	Frecuencia sugerida
Adjudicaciones públicas	Bases CONOECE / SEACE de adjudicaciones	2018-2026	Fuente principal para identificar proveedores adjudicados, montos, entidades,	Mensual o trimestral, según disponibilidad.

Tipo de dato	Fuente	Periodo	Uso dentro del proyecto	Frecuencia sugerida
			tipo de proceso y fechas de buena pro.	
Proveedores sancionados	Base de sancionados OECE	Según fecha de corte disponible	Fuente para construir la variable objetivo SANCIONADO_24 M y variables de antecedentes sancionadores.	Mensual o cada vez que el OECE actualice el registro.
RUC del proveedor	Campo ruc_proveedor en adjudicaciones y RUC en sancionados	2018-2026	Llave principal para integrar adjudicaciones y sanciones.	En cada actualización de la base.
Fecha de buena pro	Base de adjudicaciones	2018-2026	Fecha base para medir el horizonte de 24 meses de sanción futura.	En cada actualización de adjudicaciones

Tipo de dato	Fuente	Periodo	Uso dentro del proyecto	Frecuencia sugerida
Fecha de inicio de sanción	Base de sancionados	Según disponibilidad	Permite identificar si la sanción ocurrió dentro del horizonte posterior a la adjudicación.	En cada actualización de sanciones.
Monto adjudicado	Base de adjudicaciones	2018-2026	Variable predictora y base para calcular exposición económica al riesgo.	En cada actualización de adjudicaciones.
Tipo de proceso	Base de adjudicaciones	2018-2026	Variable categórica para identificar diferencias de riesgo según modalidad de contratación.	En cada actualización de adjudicaciones.
Objeto contractual	Base de adjudicaciones	2018-2026	Variable categórica para analizar riesgo según bienes, servicios, obras o consultorías.	En cada actualización de adjudicaciones.

Tipo de dato	Fuente	Periodo	Uso dentro del proyecto	Frecuencia sugerida
Entidad contratante	Base de adjudicaciones	2018-2026	Permite analizar concentración de riesgo por entidad o tipo de entidad.	En cada actualización de adjudicaciones .
Departamento	Base de adjudicaciones	2018-2026	Permite analizar patrones geográficos y segmentar el riesgo territorialmente.	En cada actualización de adjudicaciones .

La volumetría estimada del proyecto es adecuada para aplicar técnicas de ciencia de datos y machine learning. La integración de adjudicaciones y sanciones permite construir un dataset modelable con observaciones suficientes, casos positivos y negativos, así como variables explicativas de naturaleza económica, temporal, geográfica y procedimental. La frecuencia de actualización dependerá de la disponibilidad de las fuentes públicas, pero para un uso operativo se recomienda una actualización mensual o trimestral.